Proyecto 1 — Rover autónomo

Búsqueda, planificación y metaheurísticas para la exploración de una región.

Autor: Angélica María Martínez Céspedez

Asignatura: Elementos de Inteligencia Artificial ·

Resumen

Se implementó un agente autónomo capaz de explorar una región marciana modelada como una rejilla con costos de terreno, recolectar y analizar muestras científicas y transmitirlas, sujeto a restricciones de batería, memoria y ubicación. El sistema integra cuatro módulos que se conectan entre sí: **búsqueda** (BFS, DFS, Dijkstra, A* y voraz), **metaheurísticas** (Hill Climbing y Recocido Simulado) para decidir el orden de visita, **planificación** (representación simbólica elegida) con A* sobre un espacio de acciones con precondiciones, y un **sistema basado en reglas** (módulo opcional E). El generador de entornos es **parametrizable en tamaño, estructura y puntos de carga**, y **garantiza que toda instancia tenga solución**. Todo el código y los experimentos son reproducibles a partir de semillas fijas. Los resultados muestran que A* domina a Dijkstra (igual solución, ~52 % menos de nodos), que las metaheurísticas se acercan al óptimo (~3 %) con un presupuesto modesto, y que el planificador exacto es correcto pero no escala con el número de puntos de interés (lo que justifica precisamente usar metaheurísticas).

1. Descripción y modelado del problema

El rover se mueve por una **rejilla** de celdas (`width × height`). Cada celda tiene un **terreno** con un costo de entrada: llanura (1), piedra (2), roca (3), arena fina (4) y duna (5); los obstáculos son intransitables. Sobre la rejilla se ubican cuatro tipos de puntos:

- **Base**: punto de recarga y de partida.
- **Estaciones de carga**: puntos de recarga adicionales (la base ya es uno de ellos).
- **Puntos de interés (POI)**: las muestras científicas que hay que procesar.
- **Zonas de comunicación**: los únicos lugares desde donde se puede transmitir.

El rover tiene **batería** (capacidad 100, se agota con las acciones y se recarga gratis en cualquier punto de recarga) y **memoria** (capacidad 3 muestras pendientes). El estado del agente es la posición, la batería, la memoria usada y los conjuntos de muestras recolectadas, analizadas y transmitidas.

Las **acciones** son: `move` (desplazarse, consume batería igual al costo del terreno de destino), `collect` (10, ocupa un lugar de memoria), `analyze` (5, solo donde se recolectó), `transmit` (5, solo en una zona de comunicación, libera un lugar de memoria) y `recharge` (en la base o en una estación de carga, gratis, restaura la batería). La **misión** consiste en recolectar, analizar y transmitir *todas* las muestras. La restricción de memoria obliga a desviarse a transmitir cada tres muestras, y la batería obliga a planear las recargas; la combinación de ambas hace que el problema no sea un simple ruteo geográfico.

La **puntuación** de una misión es `100 · (muestras transmitidas) − costo total`, donde el costo total es la suma de los costos de las acciones.

Generador parametrizable y con solución garantizada. Las instancias se generan de forma reproducible a partir de una semilla y son parametrizables en tres ejes: **tamaño** (`width`, `height`), **estructura** (la mezcla de terrenos se elige entre los perfiles `flat`, `default` y `rough`, y la densidad de obstáculos con `obstacle_prob`) e **infraestructura** (`n_charging_stations`). Además, el generador **garantiza que la misión sea resoluble**: primero conecta todos los puntos clave (si un POI queda encerrado por obstáculos, se abre un camino) y después asegura que cada punto clave sea alcanzable **dentro de la capacidad de la batería** desde algún punto de recarga (si no, abre un corredor llano). Así ningún tablero es irresoluble por falta de energía, que era una de las preocupaciones planteadas en clase.

Se usan dos representaciones complementarias: un **grafo implícito de la rejilla** (estados = celdas, acciones = 4 vecinos) para la búsqueda de rutas, y un **grafo de puntos clave** (base, estaciones de carga, POI y zonas de comunicación) cuyas distancias se precalculan con A*, para la planificación y las metaheurísticas.

2. Módulo de búsqueda

Se implementaron cinco algoritmos sobre la rejilla: dos **no informados —BFS** (anchura, minimiza el número de pasos e ignora el terreno) y **DFS** (profundidad, encuentra *una* ruta cualquiera)—, **Dijkstra** (costo uniforme, óptimo), **A*** con la heurística de distancia Manhattan (admisible y consistente, porque el costo mínimo de entrar a una celda es 1) y **búsqueda voraz** (*greedy best-first*), que es rápida pero no óptima. Este módulo resuelve la ruta de menor costo entre dos puntos, que es la pieza que reutilizan la planificación y las metaheurísticas.

Protocolo : 5 instancias de 16×16 con 5 POI (semillas 11, 23, 37, 41, 59). En cada instancia se resuelven los 5 tramos `base → POI₁ → … → POI₅` con los 5 algoritmos: **125 corridas**. Se reporta la media y la desviación estándar (no la mejor corrida).

Algoritmo	Costo de ruta	Nodos expandidos	Tiempo (ms)
BFS	25,9 ± 15,9	104,9 ± 64,6	0,13
DFS	120,0 ± 59,9	121,6 ± 57,1	0,16
Dijkstra	**18,4** ± 10,6	99,4 ± 67,8	0,17
A*	**18,4** ± 10,6	**47,8** ± 42,6	**0,10**
Voraz	27,2 ± 17,2	12,5 ± 8,2	0,03

Análisis : Dijkstra y A* devuelven rutas del mismo costo (ambos óptimos), pero A* expande ~52 % menos nodos y es más rápido: la heurística guía la búsqueda sin perder optimalidad. El **voraz** es el más rápido y el que menos nodos explora, pero paga un **48 % más de costo**. Las búsquedas **no informadas** confirman por qué hace falta la información del terreno: **BFS** minimiza pasos, no costo, y termina un 41 % más caro que el óptimo; **DFS** es el peor con diferencia (120 frente a 18,4), porque encuentra la primera ruta que aparece sin ninguna noción de calidad. En consecuencia, **A* es la opción por defecto** en el resto del sistema.

Comparación entre estructuras : Para comprobar que los algoritmos se comportan igual en "sistemas" distintos, se repitió la búsqueda con el mismo tamaño (14×14, 5 POI) pero con tres **perfiles de estructura**: `flat` (casi todo llano), `default` y `rough` (predominan roca y duna). El costo óptimo (A*) pasa de **10,4** (`flat`) a **14,3** (`default`) y a **18,2** (`rough`): la estructura del terreno cambia el costo en un 75 %, y el orden entre algoritmos se mantiene (ver Figura A4 y Tabla B3). Esto valida el generador parametrizable: las comparaciones no dependen de un único mapa.

3. Módulo de metaheurísticas

Qué se decide y por qué : El planificador exacto (Sección 4) se vuelve intratable al crecer el número de POI: si hay n POI, el número de órdenes de visita posibles es $n!$. Por eso se ataca directamente la **permutación de visita de los POI** con metaheurísticas, y se le pasa el orden propuesto al planificador.

- **Representación:** una permutación de los POI. **Vecindad:** intercambiar dos posiciones.
- **Función objetivo:** costo estimado de recorrer los POI en ese orden. Construye la ruta respetando la

capacidad de memoria (inserta una desviación a la zona de comunicación más cercana cada 3 muestras), termina en el punto de recarga más cercano, usa las **distancias reales** (A*) y suma las acciones obligatorias de cada muestra. Está deliberadamente **alineada con el plan real**.

- **Algoritmos:** Hill Climbing (acepta el vecino solo si mejora) y Recocido Simulado (acepta

empeoramientos con probabilidad decreciente). Ambos con el **mismo presupuesto: 2000 evaluaciones**.

Protocolo : 5 instancias de 14×14 con 4 POI (mismas semillas), 8 corridas por instancia y algoritmo: **80 corridas** (semillas declaradas).

Algoritmo	Función objetivo	Costo del plan real	Tiempo (ms)
Hill Climbing	155,4 ± 13,7	168,9 ± 19,7	44,6
Recocido Simulado	**154,8** ± 13,3	**168,9** ± 20,0	**29,5**

Análisis : Con el mismo presupuesto, ambos algoritmos alcanzan una calidad estadísticamente equivalente (diferencia de ~0,4 puntos, muy inferior a la desviación), pero el Recocido Simulado es ~34 % más rápido. Se observa una **brecha de ~14 puntos (~8 %)** entre el objetivo y el costo del plan: la función objetivo ignora las desviaciones extra por batería y recarga, por lo que subestima el costo real. Se reporta esta brecha de forma explícita porque condiciona la comparación con el óptimo.

4. Módulo de planificación (representación simbólica elegida)

Se eligió **planificación clásica** en lugar de CSP. El estado del planificador es la tupla `(posición, batería, memoria usada, muestras recolectadas, analizadas, transmitidas)` y el espacio se explora con **A*** sobre acciones con precondiciones y efectos:

Acción	Precondición	Efecto	Costo
`move(d)`	batería ≥ costo del tramo	posición ← d; batería −= costo	terreno de `d`
`collect(p)`	estar en `p`, sin recolectar, memoria libre	recolectada; memoria +1	10
`analyze(p)`	estar en `p`, recolectada	analizada	5
`transmit(p)`	en zona de comunicación, `p` analizada	transmitida; memoria −1	5
`recharge`	en la base **o** en una estación de carga, batería < 100	batería ← 100	0

La **meta** es transmitir todas las muestras. Los movimientos entre puntos clave se resuelven con A* sobre la rejilla, lo que **integra el módulo de búsqueda dentro del de planificación**. La heurística del planificador es **admisible** y combina dos cotas inferiores: (i) las acciones aún obligatorias para cada muestra no transmitida, y (ii) una cota de movimiento dada por el **árbol de expansión mínimo** sobre los POI pendientes y una zona de comunicación. Al ser admisible, A* **garantiza el plan óptimo**.

Integración con las metaheurísticas : El parámetro `enforce_order=True` restringe el orden de recolección al propuesto (por la metaheurística o por las reglas): el planificador lo respeta, pero sigue decidiendo libremente cuándo recargar y cuándo transmitir. Sin este mecanismo, el planificador ignoraba el orden y las metaheurísticas no influían en la solución.

Validación : Cada plan se ejecuta en un **simulador** que comprueba las precondiciones paso a paso (que no se agote la batería, que no se exceda la memoria, que se analice en el POI y se transmita solo en zonas de comunicación). Los planes tienen entre 20 y 32 acciones; en la instancia de demostración el planificador usa explícitamente la estación de carga (`move:CHARGE_1`, `recharge`).

Limitación : El planificador exacto **no escala** con el número de POI: medimos ~0,5 s con 4 POI, ~9 s con 5 POI y ~91–141 s con 6 POI. Esta limitación es la justificación empírica del módulo de metaheurísticas y se discute en la Sección 7.

5. Módulo adicional: sistema basado en reglas (módulo E, opcional)

Se implementó además un **sistema de producción con encadenamiento hacia adelante**, con reglas como: «si hay una muestra recolectada sin analizar en la posición actual → analizar»; «si la memoria está llena → ir a la zona de comunicación más cercana y transmitir»; «si ningún destino es alcanzable con margen → ir al punto de recarga más cercano»; «si quedan POI sin recolectar → ir al más cercano». El módulo produce un plan completo sin búsqueda exhaustiva y es totalmente **explicable**.

Instancia	Misiones completadas	Costo medio
12×12 / 4 POI	100 % (20/20)	183,55 ± 27,09
14×14 / 6 POI	100 % (20/20)	291,85 ± 43,39
16×16 / 5 POI	100 % (20/20)	279,65 ± 47,49

Las reglas son rápidas (milisegundos) y **completan la misión en el 100 %** de las instancias, gracias a que la política reserva batería para llegar siempre a un punto de recarga. Su costo es mayor que el del planificador: en la demostración dan 217 frente a los 158 del óptimo. Es decir, se gana velocidad y explicabilidad a costa de eficiencia.

6. Resultados experimentales y análisis comparativo

Protocolo (igual para todos los módulos): varias instancias (varios tamaños, varios números de POI y varias estructuras), varias corridas con **semillas declaradas**, **presupuesto igual** entre los algoritmos comparados (2000 evaluaciones por corrida en las metaheurísticas) y reporte de **media y desviación**, nunca de la mejor corrida. Los resultados se guardan en `outputs/` y se resumen en `outputs/summary.json`.

Los hallazgos principales son:

(1) Búsqueda : A* obtiene la misma solución que Dijkstra con la mitad de trabajo; BFS y DFS son peores porque ignoran el costo del terreno; el voraz es rápido pero subóptimo.

(2) Estructura : El costo óptimo crece con la dureza del terreno (10,4 → 14,3 → 18,2), pero el orden entre algoritmos se mantiene: las conclusiones son robustas al "sistema" elegido.

(3) Metaheurísticas : Hill Climbing y Recocido Simulado empatan en calidad con el mismo presupuesto; el Recocido Simulado es más rápido.

(4) Planificación frente a las metaheurísticas : Con el planificador exacto como referencia (12×12, 4 POI, con una estación de carga):

Método	Costo del plan
Óptimo (A* exacto)	**137,2** ± 12,1
Recocido Simulado + planificador	141,0 ± 14,0
Hill Climbing + planificador	141,0 ± 14,0
Orden aleatorio + planificador	160,4 ± 27,1

Las metaheurísticas quedan a ~3 % del óptimo y **baten al azar en ~19 puntos**, y como el azar usa el *mismo* planificador, esa diferencia se debe únicamente a la calidad del orden. Esto confirma que la formulación del problema como permutación es adecuada.

(5) Demostración integrada (semilla 23, 12×12, 4 POI, 1 estación de carga) : Hill Climbing y Recocido Simulado coinciden en el orden `POI_4 → POI_1 → POI_2 → POI_3`; al imponerlo, el plan forzado cuesta **162** frente a los **158** del óptimo libre (una diferencia del 2,5 %), y el planificador **usa la estación de carga** para recargar a mitad de misión. El sistema de reglas gasta 217 pero también completa la misión.

7. Desafíos encontrados y cómo se resolvieron

1. **La metaheurística no influía en el plan :** La primera versión del planificador recalculaba el mejor

orden y devolvía siempre la misma solución. *Solución:* se añadió `enforce_order`, que restringe la recolección al orden propuesto.

1. **Función objetivo desalineada :** La primera versión solo sumaba distancias e ignoraba memoria y

batería, así que «mejoraba» órdenes que en el plan real eran peores. *Solución:* se alineó la función con el modelo (desviaciones por memoria, distancias reales de A* y acciones obligatorias).

1. **Presupuesto desigual :** Hill Climbing hacía menos evaluaciones que el Recocido Simulado, una

comparación injusta. *Solución:* mismo presupuesto (2000 evaluaciones) para ambos.

1. **Tableros sin solución :** Algunos POI quedaban encerrados por obstáculos y, en mapas grandes, podían

quedar a más de una batería de distancia. *Solución:* el generador ahora **garantiza conectividad y viabilidad de batería** (abre corredores si hace falta) y admite **estaciones de carga** adicionales.

1. **Rendimiento del planificador exacto :** Con 5–6 POI el A* tardaba minutos. *Solución:* se añadió una

heurística admisible más fuerte (cota de movimiento por árbol de expansión mínimo) y se ajustó el protocolo a tamaños donde el óptimo es calculable. Este hallazgo es además el argumento central a favor de las metaheurísticas.

1. **Rigor experimental :** Se fijaron semillas, se ejecutan varias corridas por instancia y se reporta la

dispersión; el protocolo completo corre en ~30 s, de modo que es fácilmente reproducible.

8. Conclusiones y posibles mejoras

Conclusiones : A* es el algoritmo de búsqueda más conveniente (óptimo y más eficiente que Dijkstra), y las búsquedas no informadas y el voraz quedan por detrás. Hill Climbing y Recocido Simulado encuentran órdenes de visita casi óptimos con un presupuesto pequeño y superan al azar usando el mismo planificador, lo que valida la descomposición del problema. El planificador simbólico es correcto y óptimo en instancias pequeñas, pero su costo crece muy rápido con el número de POI. El generador parametrizable (tamaño, estructura y puntos de carga) permite comparar los algoritmos en sistemas distintos sin perder reproducibilidad, y garantiza que todas las instancias tengan solución.

Mejoras posibles : (i) Escalar el planificador exacto con reglas de dominancia de estados o con un A* jerárquico; (ii) incorporar más metaheurísticas (búsqueda tabú, algoritmos genéticos, colonia de hormigas); (iii) añadir un módulo CSP o heurísticas tipo `hmax` / `hadd` para comparar; (iv) paralelizar las corridas; y (v) desarrollar una interfaz que anime la misión paso a paso.

Referencias

1. Russell, S.; Norvig, P. *Artificial Intelligence: A Modern Approach* (4.ª ed.). Pearson, 2021.
2. Hart, P.; Nilsson, N.; Raphael, B. *A Formal Basis for the Heuristic Determination of Minimum Cost

Paths*. IEEE Trans. Systems Science and Cybernetics, 1968.

1. Kirkpatrick, S.; Gelatt, C.; Vecchi, M. *Optimization by Simulated Annealing*. Science, 1983.
2. LaValle, S. *Planning Algorithms*. Cambridge University Press, 2006.
3. Cormen, T. et al. *Introduction to Algorithms* (3.ª ed.). MIT Press, 2009.

Apéndice A — Figuras

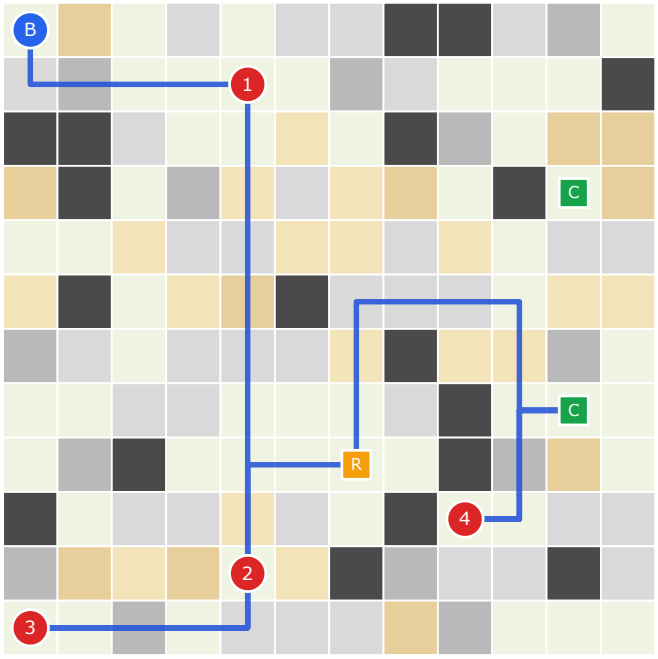


Figura A1. Entorno de la instancia semilla 23 (12×12, 4 POI, 1 estación de carga). Color = terreno; B = base; 1–4 = POI en el orden de visita; C = zona de comunicación; R = estación de carga; la línea azul es la ruta planificada (que pasa por R para recargar).

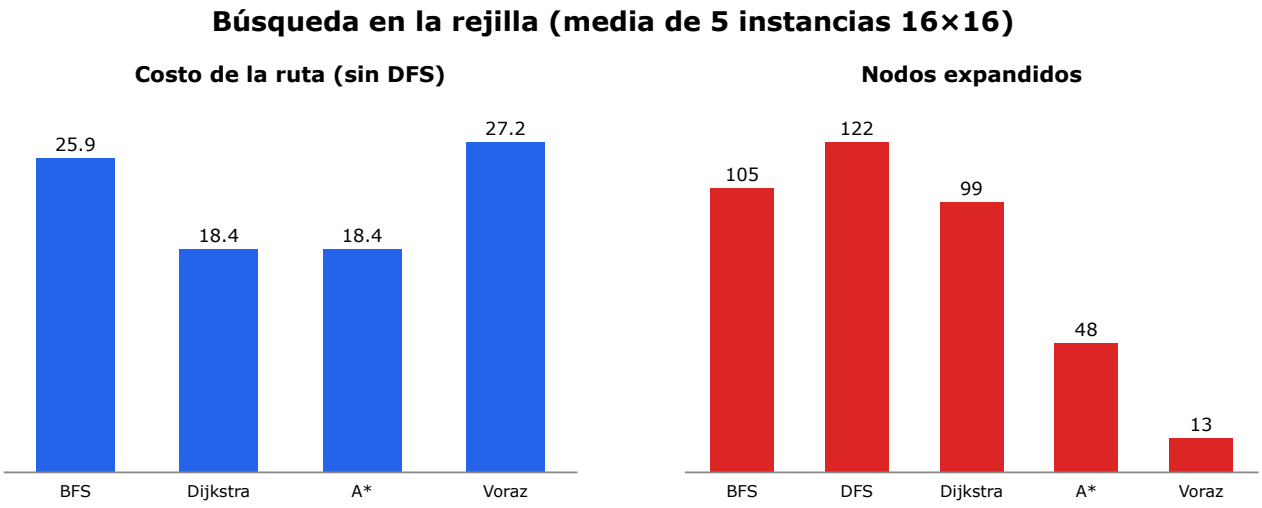


Figura A2. Comparación de los algoritmos de búsqueda: costo de la ruta (izquierda; se omite DFS por su magnitud, 120) y nodos expandidos (derecha).

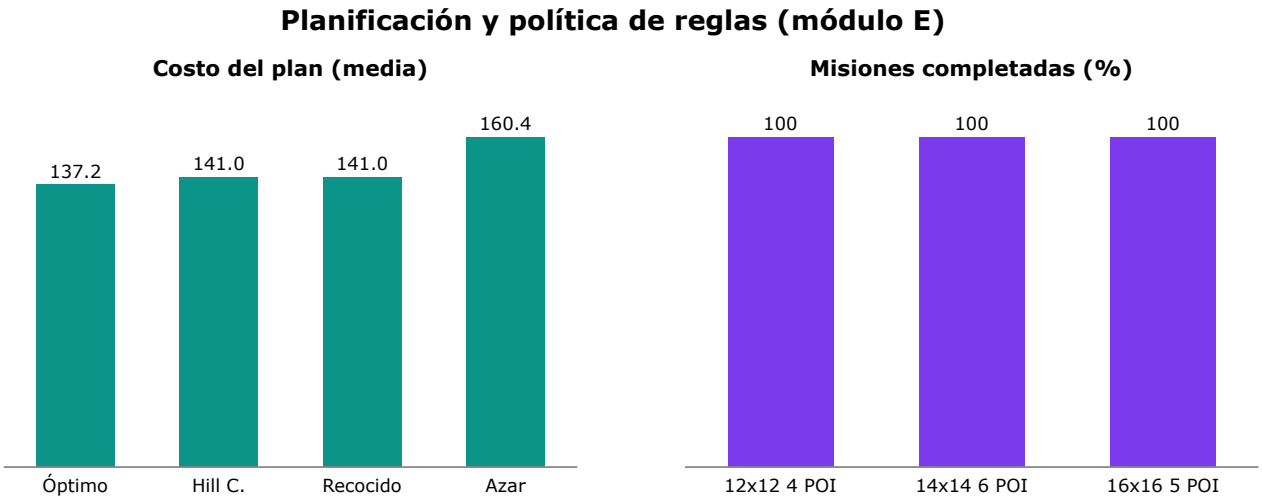


Figura A3. Costo del plan por método (izquierda) y porcentaje de misiones completadas por la política de reglas (derecha).

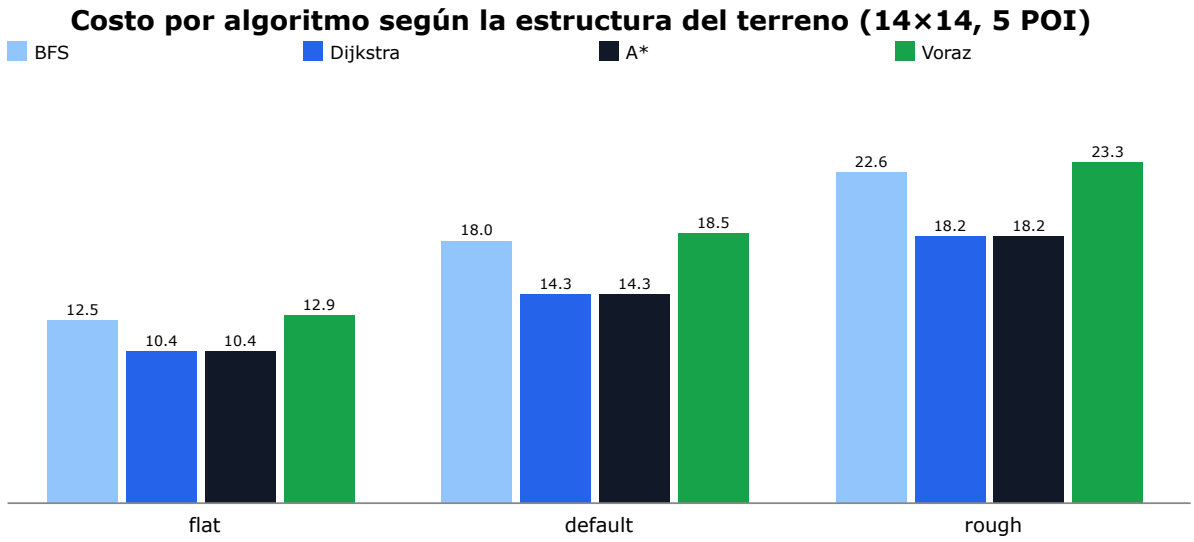


Figura A4. Costo medio por algoritmo según la estructura del terreno (perfiles flat / default / rough, 14×14 y 5 POI).

Apéndice B — Tablas

Tabla B1. Protocolo experimental.

Módulo	Instancias	Semillas	Corridas	Presupuesto
Búsqueda	16×16, 5 POI	11, 23, 37, 41, 59	5 × 5 × 5 alg. = 125	exhaustivo
Estructuras	14×14, 5 POI (3 perfiles)	11, 23, 37, 41, 59	3 × 5 × 5 × 5 alg. = 375	exhaustivo
Metaheurísticas	14×14, 4 POI	11, 23, 37, 41, 59	5 × 8 × 2 alg. = 80	2000 evaluaciones c/u
Planificación	12×12, 4 POI, 1 carga	11, 23, 37, 41, 59	5 × 9 = 45	exacto
Reglas (E)	12×12/4, 14×14/6, 16×16/5 (1–2 cargas)	1…20	3 × 20 = 60	heurístico

Tabla B2. Búsqueda, 16×16 y 5 POI (media ± desviación).

Algoritmo	Costo	Nodos expandidos	Tiempo (ms)
BFS	25,92 ± 15,91	104,92 ± 64,63	0,13
DFS	120,00 ± 59,94	121,56 ± 57,09	0,16
Dijkstra	18,40 ± 10,55	99,40 ± 67,79	0,17
A*	18,40 ± 10,55	47,76 ± 42,63	0,10
Voraz	27,24 ± 17,24	12,52 ± 8,18	0,03

Tabla B3. Costo medio por estructura de terreno (14×14, 5 POI).

Perfil	BFS	DFS	Dijkstra	A*	Voraz
flat	12,52	47,12	10,40	10,40	12,88
default	17,96	68,68	14,28	14,28	18,48
rough	22,64	85,48	18,24	18,24	23,32

Tabla B4. Metaheurísticas, 14×14 y 4 POI (media ± desviación).

Algoritmo	Objetivo	Costo del plan	Tiempo (ms)
Hill Climbing	155,40 ± 13,69	168,90 ± 19,65	44,55
Recocido Simulado	154,80 ± 13,32	168,90 ± 19,99	29,50

Tabla B5. Planificación, 12×12 y 4 POI con 1 estación de carga (media ± desviación).

Método	Costo del plan
Óptimo (A* exacto)	137,20 ± 12,12
Recocido Simulado + planificador	141,00 ± 13,96
Hill Climbing + planificador	141,00 ± 13,96
Orden aleatorio + planificador	160,43 ± 27,11

Tabla B6. Política basada en reglas (20 instancias por configuración).

Instancia	Éxito	Costo medio
12×12 / 4 POI	100 %	183,55 ± 27,09
14×14 / 6 POI	100 %	291,85 ± 43,39
16×16 / 5 POI	100 %	279,65 ± 47,49